PROJECT REPORT

FOR

Foundations of Artificial Intelligence

DuckPersona: An AI-Powered Intelligent Code Debugging Assistant

A PROJECT REPORT

Submitted by

2116240701314 – MITHULESH.S

2116240701364 – NISHANTH.B



October, 2026

BONAFIDE CERTIFICATE

Certified that this project report "DuckPersona: An AI-Powered Intelligent Code Debugging Assistant" is the bonafide work of "MITHULESH.S & NISHANTH.B" who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	3
1	INTRODUCTION	5
2	LITERATURE REVIEW	7
3	PROPOSED SYSTEM	9
	3.1 ARCHITECTURE DIAGRAM	11
4	MODULES DESCRIPTION	12
	4.1 MODULE 1	12
	4.2 MODULE 2	14
5	IMPLEMENTATION AND RESULTS	15
	5.1 EXPERIMENTAL SETUP	15
	5.2 RESULTS	17
6	CONCLUSION AND FUTURE WORK	20
7	REFERENCES	21

ABSTRACT:

DuckPersona is an AI-powered coding and debugging platform designed to help students and beginner programmers improve their programming skills. It provides an interactive environment for writing, compiling, and executing code. The platform also provides AI-based assistance for understanding programming errors. It aims to make coding and debugging easier and more accessible for users.

The system integrates a code editor, compiler, error detection, and AI chatbot into a single web application. Users can enter their code and execute it to obtain the output or identify errors. When an error occurs, the AI assistant helps explain the cause of the error. It also provides useful guidance to help users debug their code.

The **A* Search Algorithm** is incorporated into DuckPersona to support intelligent and efficient debugging analysis. It uses the actual cost and estimated cost to identify a suitable path while analyzing programming problems. By combining the cost function with a heuristic value, A* helps the system prioritize relevant debugging paths and reduce unnecessary search. This enhances the intelligent debugging process and supports the AI assistant in providing meaningful explanations and suggestions to users.

The project demonstrates the practical use of Artificial Intelligence in programming education and debugging. In the future, the system can be enhanced with support for multiple programming languages and advanced code analysis. Features such as coding history and personalized learning recommendations can also be added. DuckPersona aims to provide a simple and effective platform for learning, coding, and debugging.

Keywords: Artificial Intelligence, Code Debugging, AI Assistant, Online Compiler, Error Detection, Programming Education, Web Application.

1. INTRODUCTION

Programming plays an important role in computer science and software development. However, beginners often face difficulties in writing code, understanding compiler errors, and debugging programs. Traditional programming environments mainly display error messages, which may not clearly explain the actual cause of the problem.

The main objective of DuckPersona is to provide intelligent assistance during the programming and debugging process. Instead of depending only on compiler messages or external resources, users can communicate directly with the AI assistant. This helps beginners understand programming errors, improve their problem-solving skills, and learn from their mistakes.

Key Features of DuckPersona:

- **AI-Powered Chatbot:** Provides interactive assistance for programming-related queries and errors.
- **Code Editor:** Allows users to write and edit their source code within the platform. Enables users to compile and execute programs directly.
- **Error Detection:** Identifies compilation and execution errors in the submitted code.
- **AI-Based Debugging:** Explains errors and provides guidance for solving programming problems.
- **User Authentication:** Provides secure login and personalized access to the platform.
- **Interactive Interface:** Combines coding, execution, and AI assistance in a single environment.

DuckPersona provides a web-based environment where users can write, compile, and execute their programs. The platform combines a code editor, compilation system, error detection, and an AI-powered chatbot in one application. When an error occurs, users can interact with the AI assistant to understand the problem and receive suitable debugging guidance.

DuckPersona is mainly designed for students, beginners, and users who want to practice programming with intelligent assistance. By integrating an online coding environment with AI-based debugging, the platform reduces the difficulty of understanding programming errors and provides a more interactive learning experience.

A* Search Algorithm: DuckPersona also incorporates the A* (A-Star) Search Algorithm to support intelligent debugging analysis. It uses the actual cost and estimated cost to search for the most relevant path among possible error causes and debugging solutions. This helps the system prioritize promising solutions and provide more efficient and meaningful debugging assistance to the user.

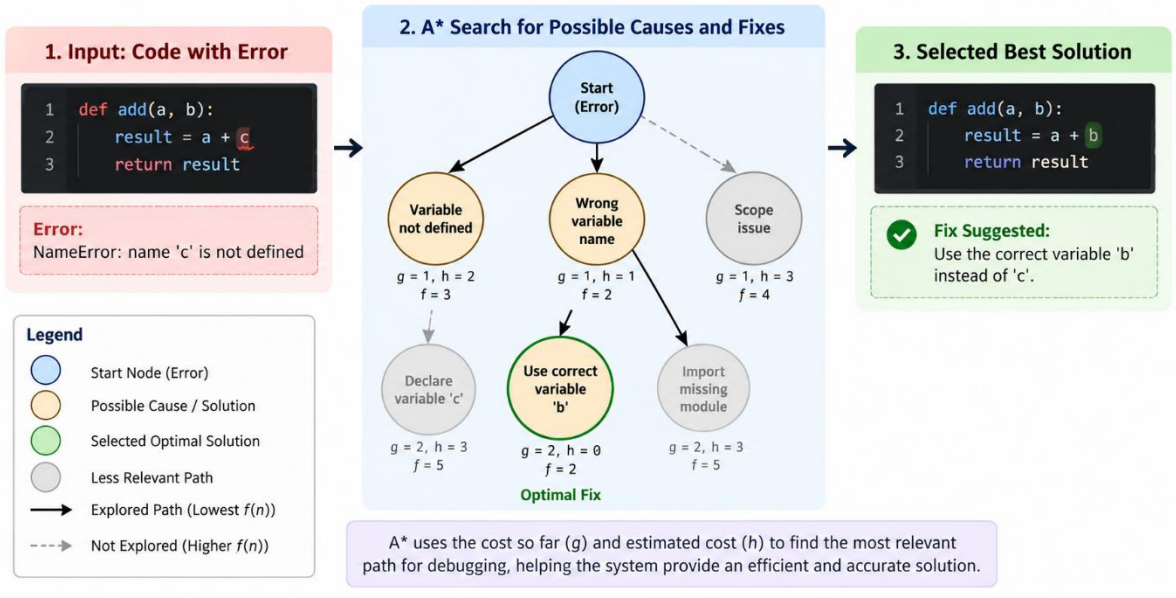


Fig 1.1 How A* Search Actually helps in project

2. LITERATURE REVIEW

AI-Based Coding Assistant Papers:

Chen et al. (2021) studied the use of artificial intelligence in programming assistance and found that AI-based systems can help users understand programming concepts and identify coding errors. The study highlighted that interactive assistance can improve the learning experience of beginner programmers. This insight influenced the integration of an AI chatbot in DuckPersona to provide programming guidance.

Li and Wang (2022) analyzed intelligent code assistance systems and observed that automated suggestions can reduce the difficulty of identifying and correcting programming errors. Their research emphasized the importance of providing explanations along with suggested solutions. This concept influenced the AI-based debugging and error explanation features of DuckPersona.

Kumar et al. (2020) studied online programming environments and found that integrated code editors and execution systems provide a convenient platform for students to practice programming. The study suggested that users benefit when writing, compiling, and testing code are available within a single interface. This finding influenced the development of the integrated coding and compilation environment in DuckPersona.

Automated Debugging and Programming Education:

Singh and Sharma (2021) investigated automated debugging techniques and found that compiler messages alone may not be sufficient for beginners to understand programming errors. The research emphasized the need for simple explanations and guidance during the debugging process. This approach influenced DuckPersona's feature of explaining errors through an AI-powered conversational interface.

Brown et al. (2022) examined the use of conversational AI in programming education and found that interactive question-and-answer systems can support students while learning programming. The study highlighted the importance of allowing learners to ask questions and receive immediate assistance. This concept was incorporated into DuckPersona's AI chatbot functionality.

Web-Based Development Platforms:

Patel and Verma (2020) studied web-based development environments and observed that combining multiple programming tools in one interface can improve accessibility and usability. Their research emphasized simple navigation and an organized workflow for coding activities. This finding influenced the design of DuckPersona as a unified platform containing the editor, compiler, debugging support, and AI assistant.

GitHub Copilot (2022) demonstrated the practical use of AI-assisted programming by providing code suggestions and programming assistance within development environments. Its implementation showed how AI can be integrated into software development workflows. This served as a reference for incorporating intelligent programming assistance into DuckPersona while focusing on error understanding and debugging.

Replit (2023) provided an online development environment that allows users to write, execute, and test programs through a web interface. The platform demonstrated the usefulness of browser-based programming environments for learning and development. This concept influenced the online coding and execution workflow implemented in DuckPersona.

3. PROPOSED SYSTEM

The proposed system, DuckPersona, is designed as an intelligent programming environment that combines conventional coding facilities with Artificial Intelligence. The system provides users with a single platform to write programs, execute them, identify errors, and obtain AI-based assistance. The primary focus is to simplify the debugging process and help users understand programming problems while they are working on their code.

The system follows an interactive workflow between the user, coding environment, compiler, and AI assistant. Initially, the user accesses the platform through the authentication system and enters the required code in the editor. The submitted program is processed by the compilation and execution component. The generated output or error information is then displayed to the user for further analysis.

When a programming error is encountered, DuckPersona provides an additional layer of assistance through its AI chatbot. The user can interact with the chatbot and provide the relevant code or error details. The AI analyzes the available information and generates an understandable response that explains the problem and suggests possible debugging steps. This allows the user to learn from the error instead of relying only on the technical compiler message.

DuckPersona can also be expanded with additional programming languages, persistent code storage, coding history, advanced code analysis, and personalized learning features. Such enhancements can improve the functionality of the platform and provide users with a broader environment for programming development and learning.

Major Components of the Proposed System

- Authentication Module – Handles user login and secure access to the platform.
- Code Editor Module – Provides an interface for writing and modifying source code.
- Compilation Module – Processes the submitted program and performs compilation and execution.
- Error Handling Module – Captures and displays compilation or execution errors.
- AI Assistant Module – Analyzes programming problems and provides explanations and debugging guidance.
- Chat Interface Module – Enables users to communicate with the AI assistant through a conversational interface.

A* Search Algorithm: Uses heuristic-based path searching to identify an efficient debugging path by prioritizing the most relevant code issues and reducing unnecessary search during error analysis.

The overall system is designed to maintain a simple interaction between coding and intelligent assistance. Users do not need to move between multiple applications to test their code and seek help. The combination of the editor, compiler, error handling mechanism, and AI chatbot creates a unified workflow for programming practice and debugging.

3.1 ARCHITECTURE DIAGRAM

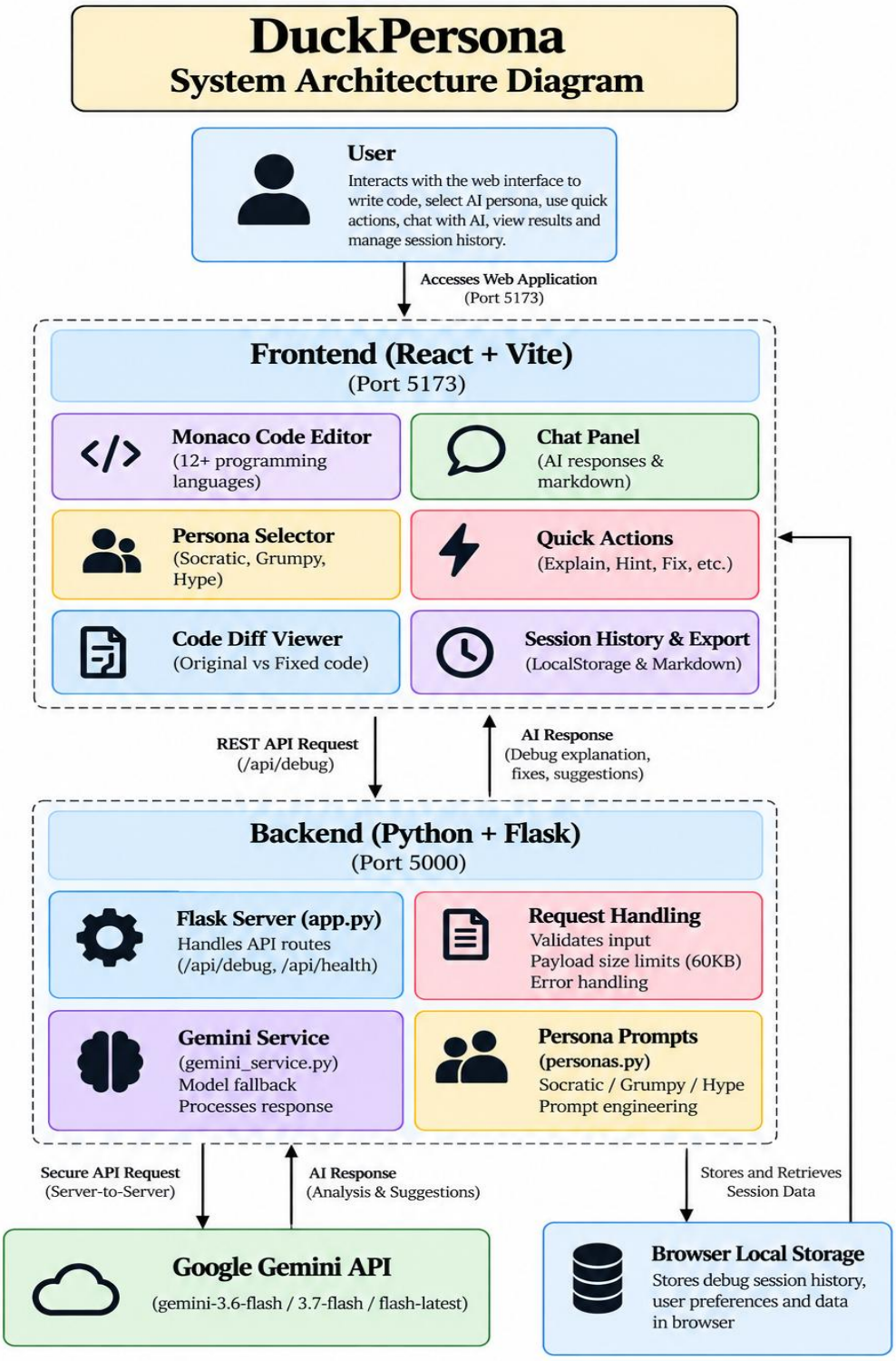


Figure 3.1: System Architecture of DuckPersona

4. MODULES DESCRIPTION

DuckPersona is organized into two major modules that work together to provide an interactive AI-based debugging environment. The first module handles the user interface and coding activities, while the second module manages AI processing, debugging analysis, and persona-based responses. The integration of these modules allows users to write code, identify problems, and receive personalized debugging assistance.

4.1 MODULE 1 – CODING AND USER INTERFACE MODULE

The Coding and User Interface Module forms the primary interaction layer of DuckPersona. It provides a simple and interactive workspace where users can write, edit, and work with their programming code. The module is developed using React and Vite and uses the Monaco Editor to provide a professional coding experience with syntax highlighting and support for multiple programming languages.

The module provides users with an integrated debugging workspace where they can enter their code and view the execution output or error information. Users can select one of the three available AI personas: Socratic Professor, Grumpy Senior Developer, or Hype Cheerleader. The selected persona determines the style in which the AI communicates during the debugging process.

An interactive chat panel is included to allow users to communicate with the AI debugging assistant. Quick actions such as **Explain Simpler, Show the Bug, Give Me a Hint, Show Corrected Code, Explain Line by Line, Find Edge Cases, and Optimize This Code** help users perform common debugging tasks quickly. The Code Diff Viewer also allows users to compare the original code with the AI-suggested corrected code.

The module also provides session management features that allow debugging conversations to be stored locally in the browser. Users can search and resume previous debugging sessions and export complete sessions in Markdown format. These features make the coding environment more organized and allow users to review their previous debugging activities and learning progress.

Main functions:

- Code writing and editing using Monaco Editor.
- Code execution and error viewing.
- AI persona selection.
- Interactive debugging chat.
- Quick debugging actions.
- Original and corrected code comparison.
- Local debugging session history.
- Markdown export of debugging sessions.

4.2 MODULE 2 – AI DEBUGGING AND BACKEND MODULE

The AI Debugging and Backend Module is responsible for processing debugging requests and communicating with the Google Gemini API. The Python Flask backend receives requests from the frontend through the REST API and validates the submitted information before processing it. The Gemini service analyzes the code, error details, and selected persona to generate a suitable debugging response.

The module supports three different AI personas: Socratic Professor, Grumpy Senior Developer, and Hype Cheerleader. Each persona uses a different prompting approach to provide debugging assistance in its own style. The backend also handles API communication, model fallback, error handling, and secure processing of the Gemini API key. The generated response is then sent back to the frontend and displayed in the chat interface.

Main functions:

- REST API request handling.
- Input validation and error handling.
- Communication with Google Gemini API.
- Persona-based prompt processing.
- AI-powered code analysis and debugging.
- Generation of explanations, hints, and fixes.
- Secure API key handling.
- Returning AI responses to the frontend.

5. IMPLEMENTATION AND RESULTS

The DuckPersona system was implemented as a web-based AI debugging application that combines a modern coding interface with an AI-powered debugging assistant. The frontend was developed using React with Vite, while the backend was implemented using Python and Flask. The application connects the frontend and backend through REST API communication, allowing debugging requests and AI-generated responses to be exchanged efficiently.

5.1 EXPERIMENTAL SETUP

The frontend of DuckPersona was developed using React and Vite with Tailwind CSS for the user interface. The Monaco Editor was integrated to provide a professional code editing environment with support for multiple programming languages. The frontend also contains the persona selector, chat panel, quick actions, code diff viewer, session history, and Markdown export functionality.

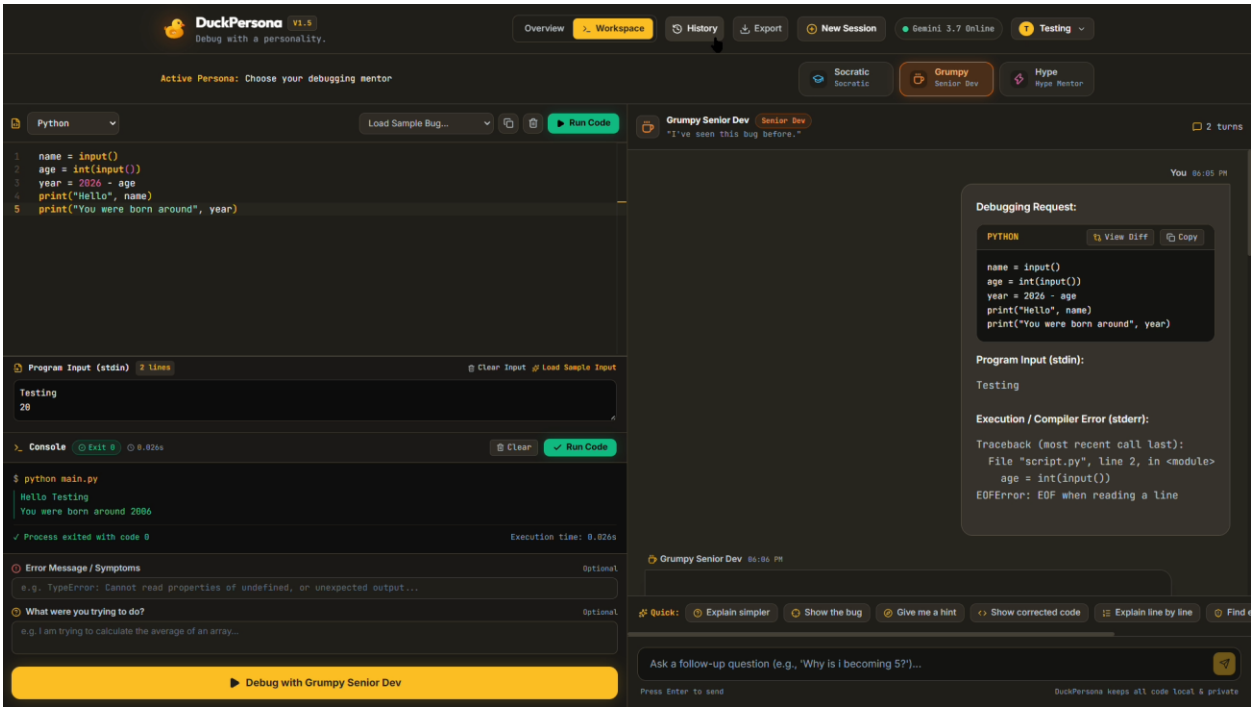


Figure 5.1.1 – DuckPersona Frontend Workspace Code Area.

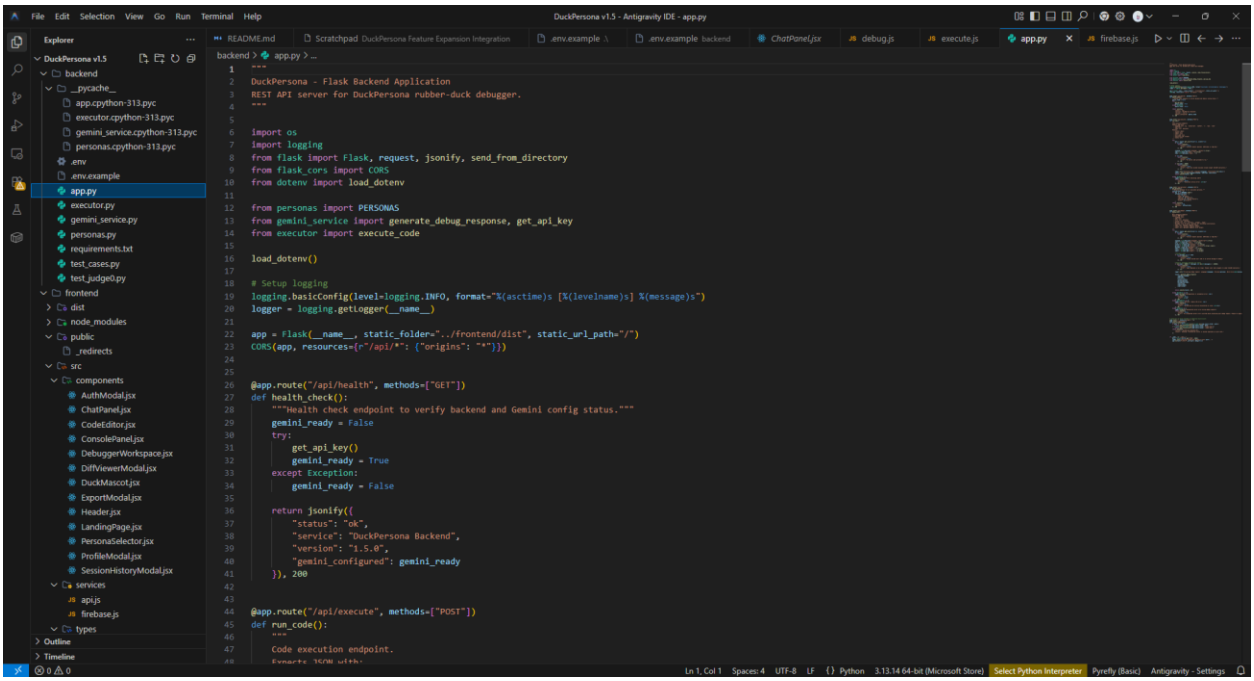


Figure 5.1.2 – DuckPersona Backend Implementation.

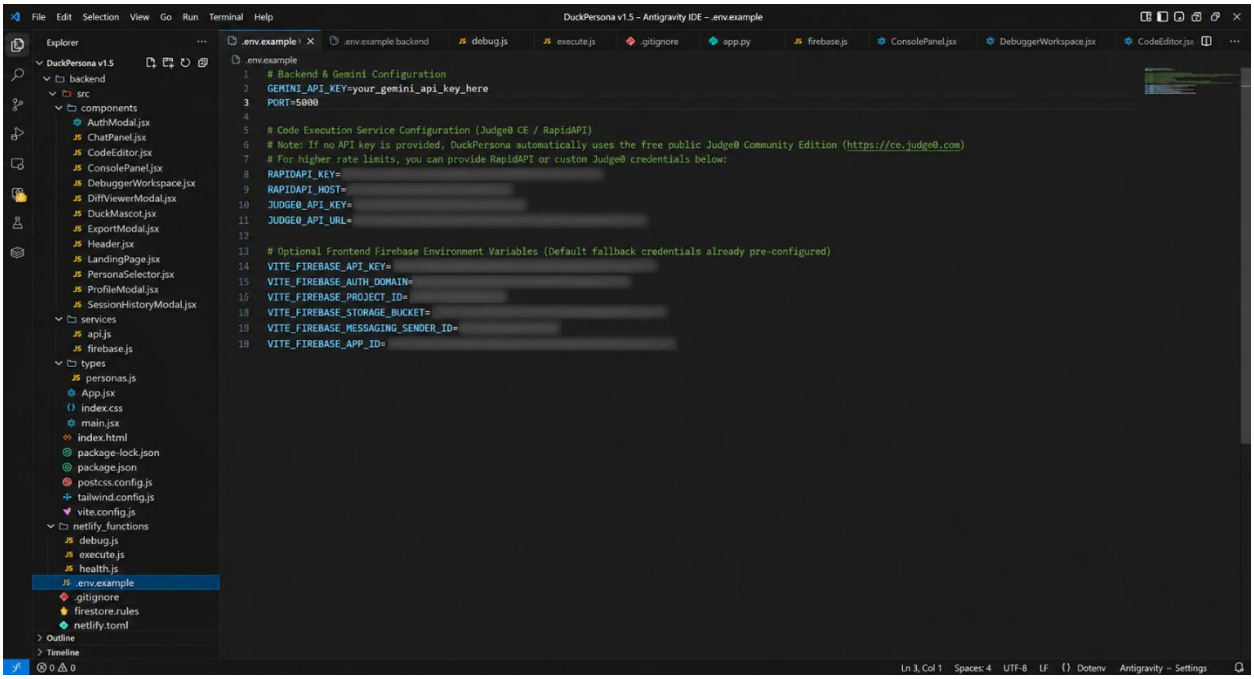


Figure 5.1.3 – Gemini API and Firebase Integration Keys.

5.2 RESULTS

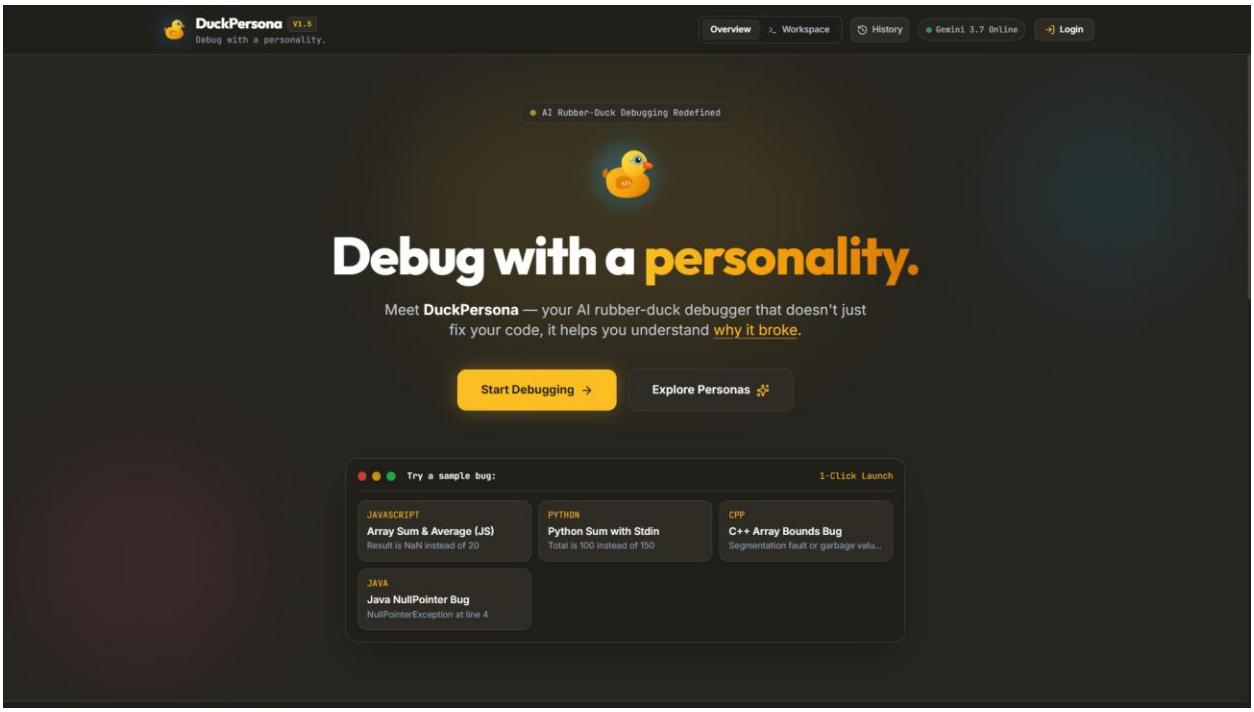


Figure 5.2.1 DuckPersona Homepage

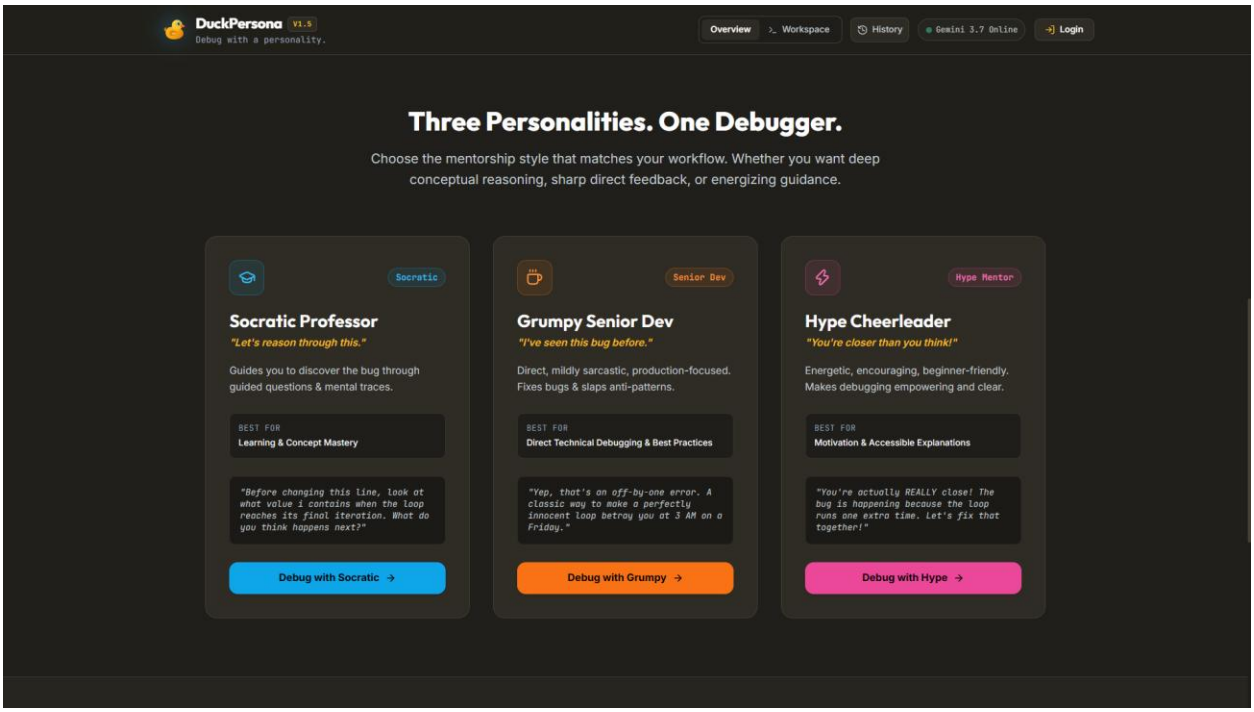


Figure 5.2.2 Three Personalities for Debugging

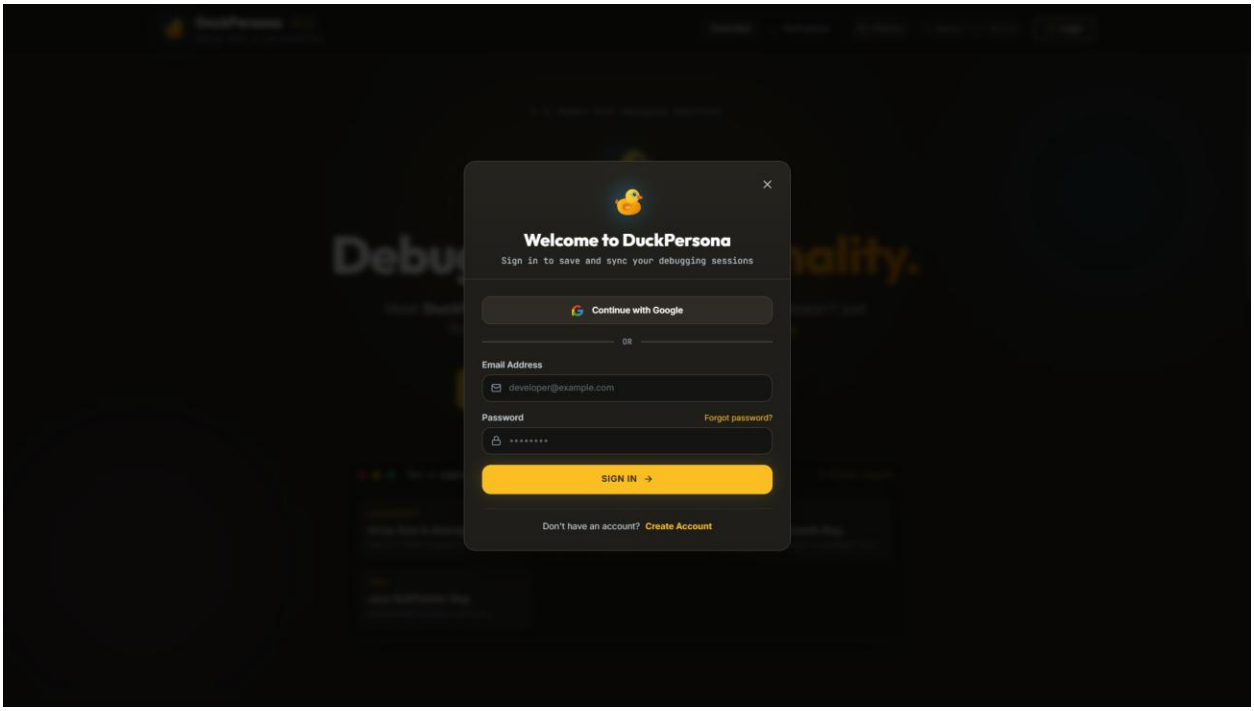


Figure 5.2.3 Login page with Google Authentication

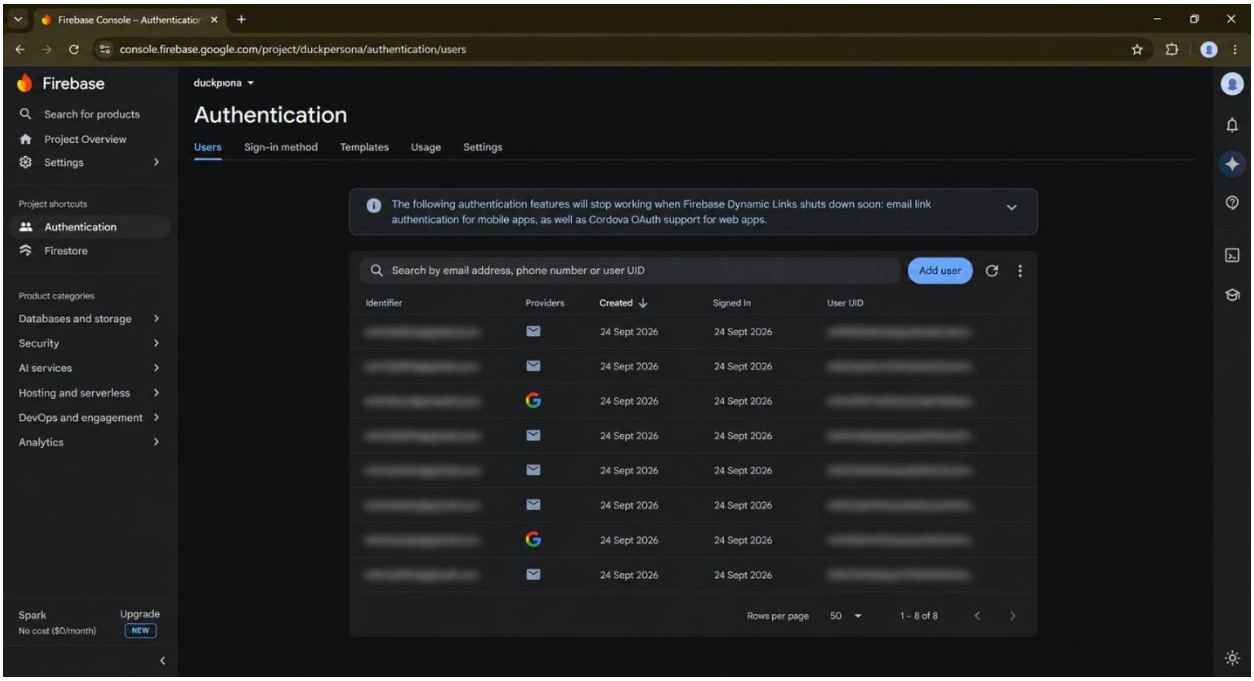


Figure 5.2.4 Firebase Authentication Centre

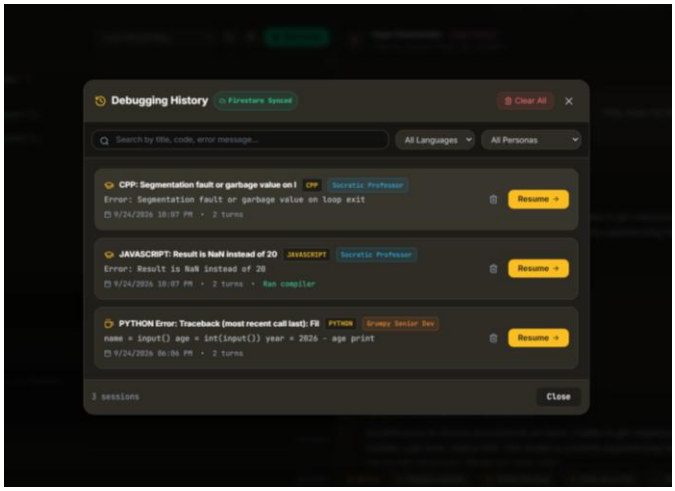


Fig 5.2.5 User Based History Option

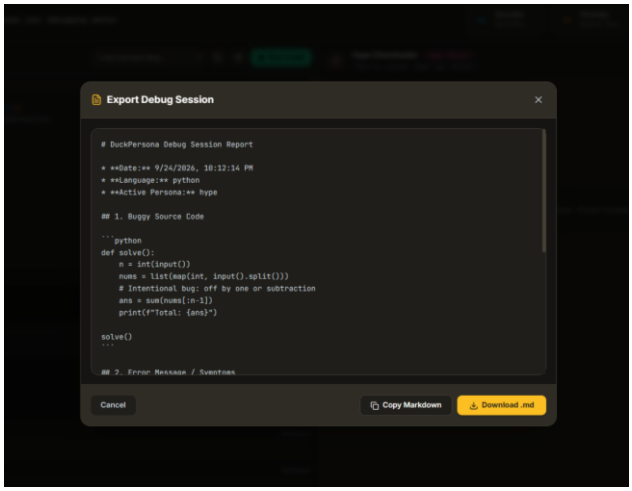


Fig 5.2.6 Export Session in .md

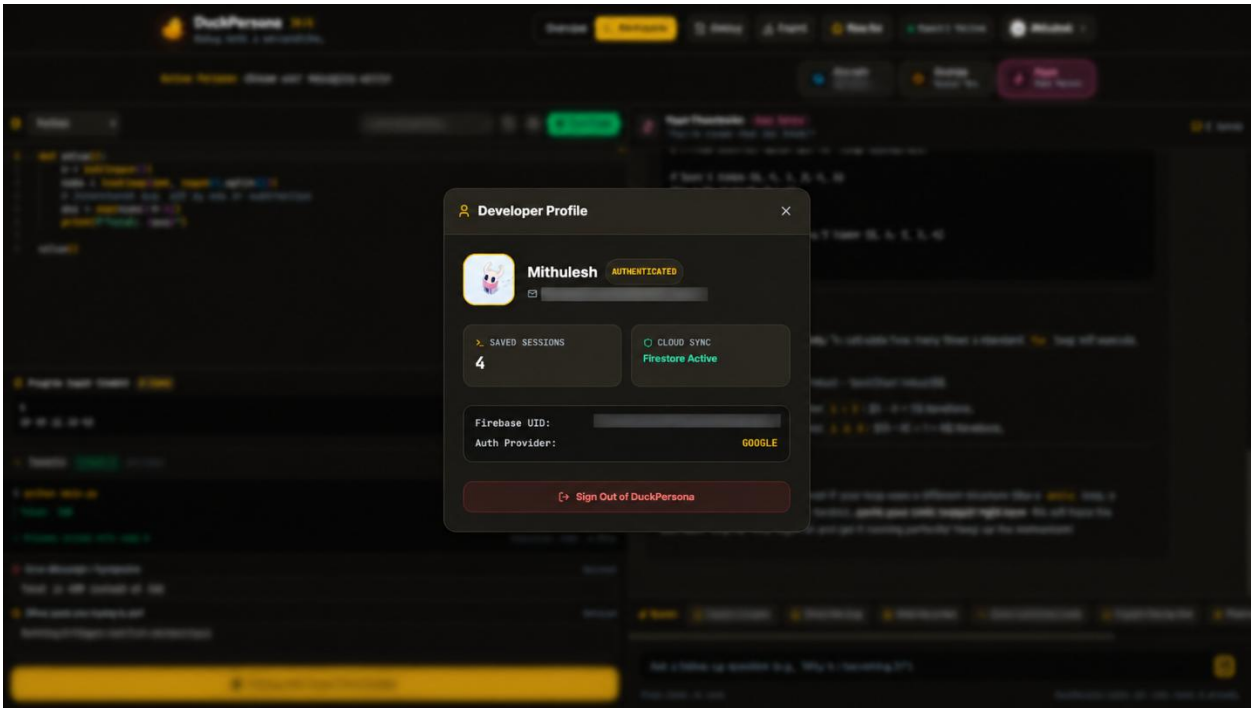


Figure 5.2.7 Profile Details linked with Firebase

6. CONCLUSION AND FUTURE WORK

DuckPersona is an AI-powered debugging platform developed to make programming error analysis more interactive and understandable. The system combines a Monaco-based code editor, code execution, AI chatbot, Firebase authentication and session management, and multiple debugging personas. The integration of the **A* Search Algorithm** provides an intelligent search approach for identifying an efficient path during the debugging process. By combining these technologies, DuckPersona helps users understand programming errors instead of simply providing a corrected solution.

The system was implemented using React, Vite, Python, Flask, Firebase, and Google Gemini API. The three AI personas, namely Socratic Professor, Grumpy Senior Developer, and Hype Cheerleader, provide different styles of debugging assistance. Features such as quick actions, code comparison, session history, and Markdown export further improve the overall debugging experience. The developed system was successfully tested in a local environment using different programming errors and AI-based debugging interactions.

FUTURE WORK

In the future, DuckPersona can be enhanced by supporting more programming languages and advanced code analysis techniques. The **A*** based debugging approach can also be improved to analyze larger and more complex debugging paths. Additional features such as automated test-case generation, real-time code suggestions, performance analysis, and personalized learning recommendations can be integrated to make the system more powerful.

The platform can also be extended with improved debugging history, cloud-based synchronization, advanced error prediction, and integration with development environments. These enhancements can help DuckPersona evolve from an AI debugging assistant into a more comprehensive intelligent programming and learning platform.

7. REFERENCES

1. **Google AI for Developers** (2025) 'Gemini API Documentation', Google, Available at: Google AI for Developers.
2. **Hart, P.E., Nilsson, N.J. and Raphael, B**. (1968) 'A Formal Basis for the Heuristic Determination of Minimum Cost Paths', *IEEE Transactions on Systems Science and Cybernetics*, Vol. 4, No. 2, pp. 100–107.
3. **Meta Platforms, Inc.** (2024) 'React Documentation', Meta Open Source, Available at: React Documentation.
4. **Microsoft** (2024) 'Monaco Editor Documentation', Microsoft, Available at: Monaco Editor Documentation.
5. **Pallets** (2024) 'Flask Documentation', Pallets Projects, Available at: Flask Documentation.
6. **Replit** (2024) 'Online Coding and Development Environment', Replit, Available at: Replit Documentation.
7. **Russell, S. and Norvig, P.** (2021) *Artificial Intelligence: A Modern Approach*. 4th edn. Pearson.
8. **Tailwind Labs** (2024) 'Tailwind CSS Documentation', Tailwind Labs, Available at: Tailwind CSS Documentation.
9. **Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł. and Polosukhin, I.** (2017) 'Attention Is All You Need', *Advances in Neural Information Processing Systems*, Vol. 30.
10. **Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. and Cao, Y.** (2023) 'ReAct: Synergizing Reasoning and Acting in Language Models', *International Conference on Learning Representations (ICLR)*.